

I. CHIẾN LƯỢC TỔNG QUAN

1. **The Hook (3 phút):** Làm hội đồng thấy sợ về vấn đề lộ Key -> Khao khát giải pháp.
 2. **The Solution & Engineering (10 phút):** "Secret Tea" là gì và nó hoạt động thông minh thế nào (trọng tâm).
 3. **The Proof (5 phút):** Demo, đánh giá và kết luận.
-

II. CHI TIẾT TỪNG SLIDE

Phần 1: Mở đầu & Đặt vấn đề

Slide 1: Trang bìa

- **Nội dung:** Tên đề tài, Tên SV, GVHD. Logo trường & Logo "Secret Tea".
- **Lời dẫn:** Chào hội đồng đồng dạc.

Slide 2: Thực trạng - "The Silent Killer"

- **Visual:** Show các bài báo về vụ lộ key (Uber, AWS mining, Samsung...). Một biểu đồ nhỏ về xu hướng tấn công chuỗi cung ứng (Supply Chain Attack).
- **Key Message:**
 - "Lộ secret không phải là vấn đề 'có thể' xảy ra, mà là 'khi nào' sẽ xảy ra."
 - Hậu quả: Mất tiền, mất uy tín, mất dữ liệu.

Slide 3: Tại sao các công cụ hiện tại chưa đủ? (Gap Analysis)

- **Visual:** Bảng so sánh hoặc hình ảnh Developer đang bối rối trước màn hình đen thui của Terminal.
- **Nội dung:**
 - *Server-side scanning (Gitleaks CI):* Quá trễ (Code đã bị push, đã lưu vào git history).
 - *Client-side CLI:* Khó dùng, phải nhớ lệnh, không trực quan.
 - **Vấn đề cốt lõi:** Developer lười bảo mật vì nó bất tiện.

Slide 4: Giải pháp "Secret Tea" & Tiếp cận Shift-left

- **Visual:** Sơ đồ dòng thời gian DevOps. Mũi tên trở ngược về phía trái (Code/IDE).

- **Key Message:**
 - Đưa bảo mật vào ngay IDE (VS Code).
 - Chặn lỗ hổng **trước khi** bấm git commit.
 - Bảo vệ thị giác (Visual Security) khi làm việc từ xa (Masking).
-

Phần 2: Giải pháp & Kỹ thuật (Phần quan trọng nhất - Show kiến thức Architect)

Slide 5: Kiến trúc Hệ thống (System Architecture)

- **Visual: Sơ đồ khối (Block Diagram).**
 - *VS Code Extension Host <-> Adapter Layer <-> Gitleaks Binary.*
 - *VS Code Editor API* (cho việc Masking).
- **Lời dẫn (Quan trọng):**
 - "Em sử dụng kiến trúc **Hexagonal (Ports & Adapters)** để tách biệt logic scan core và giao diện VS Code. Điều này giúp hệ thống dễ bảo trì."
 - Nhấn mạnh việc chạy tiến trình con để không làm đơ UI của IDE.

Slide 6: Kỹ thuật Phát hiện (Detection Engine)

- **Visual:** Hình minh họa Regex đơn giản.
- **Nội dung:**
 - Sử dụng Gitleaks Engine (Go-based) vì tốc độ vượt trội.
 - Cơ chế Entropy để phát hiện mật khẩu ngẫu nhiên.
 - Regex cho các structured keys (AWS, Google).

Slide 7: Tính năng "Killer": Auto-Masking & Privacy

- **Visual:** Ảnh GIF so sánh: Trước khi bật (hiện rõ key) vs. Sau khi bật (hiện ****).
- **Kỹ thuật (Deep Dive):**
 - Giải thích việc dùng `TextEditorDecorationType` của VS Code API.
 - **Điểm nhấn:** "Em không sửa đổi nội dung file gốc (tránh làm hỏng code), em chỉ phủ một lớp 'mặt nạ' ảo lên trên. Đây là kỹ thuật khó nhất trong dự án."

Slide 8: Tối ưu hóa & Build System

- **Visual:** Sơ đồ CI/CD Github Actions matrix build (Win, Mac, Linux).
- **Nội dung:**

- Vấn đề: Binary đa nền tảng rất nặng.
- Giải pháp: Platform-specific builds.
- Kết quả: Giảm dung lượng extension từ 50MB -> 5MB (Ví dụ).

Slide 9: Bảo mật chuỗi cung ứng (NPM Security)

- **Visual:** Quy trình scan package.json.
 - **Nội dung:**
 - Check version & tên package với Database Malware (Aikido).
 - Cảnh báo ngay khi npm install.
-

Phần 3: Kết quả & Demo

Slide 10: Demo (Video)

- **Lưu ý:** Đừng demo live nếu mạng không ổn định. Hãy quay video màn hình cắt dựng gọn gàng (tăng tốc độ x1.5).
- **Kịch bản Demo:**
 1. Gỡ một AWS Key -> Extension cảnh báo ngay.
 2. Bật chế độ "Streamer Mode" -> Key biến thành ****.
 3. Thêm một thư viện độc hại vào package.json -> Cảnh báo đỏ.
 4. Cố tình commit key -> Git hook chặn lại.

Slide 11: Đánh giá Hiệu năng & Kiểm thử

- **Visual:** Biểu đồ cột.
- **Số liệu:**
 - Thời gian scan 1 file: < 50ms.
 - Thời gian scan cả repo (1000 files): ... giây.
 - Tỷ lệ phát hiện chính xác (trên tập dataset mẫu).

Slide 12: Hạn chế & Hướng phát triển

- **Thành thật:** "Hiện tại chưa có Dashboard quản lý tập trung."
- **Tương lai:** Tích hợp AI để giảm False Positive, hỗ trợ thêm Python/Java (Maven/Pip).

Slide 13: Kết luận & Cảm ơn

- Khẳng định lại giá trị: "Secret Tea không chỉ là công cụ, nó là người vệ sĩ thầm lặng cho Developer."
-

Pro Tips

1. **Quy tắc "Tam giác":** Khi nói về một tính năng, hãy theo quy trình: **Vấn đề** (Tại sao cần?) -> **Giải pháp** (Em làm gì?) -> **Kết quả** (Nó chạy thế nào?).
2. **Chuẩn bị cho câu hỏi khó:**
 - *Hỏi:* "Tại sao dùng Gitleaks mà không tự viết engine?"
 - *Đáp:* "Dạ thưa thầy, trong kỹ thuật phần mềm, nguyên lý '**Don't Reinvent the Wheel**' rất quan trọng. Gitleaks là chuẩn công nghiệp, đã được tối ưu cực tốt bằng Go. Việc em tận dụng và build wrapper xung quanh nó để tăng trải nghiệm người dùng (UX) mang lại giá trị thực tế cao hơn là cố viết lại một engine kém hiệu quả hơn." (Trả lời thế này là 10 điểm tư duy Architect).